NexLearn LMS

Sequence Diagram Document

UE23CS352B — Object Oriented Analysis & Design
Domain: Education | Technology: Spring Boot 3.4 + Next.js 15 + MongoDB

Team Members	Laharish S (PES2UG23CS298)
	Kathit Joshi (PES2UG23CS264)
	Kavyansh Jain (PES2UG23CS268)
	Kumarchandra Edupuganti (PES2UG23CS292)

1. Document Overview

This document presents the Sequence Diagrams for all 10 use cases of the NexLearn Learning Management System. Each sequence diagram models the object-level message exchange between actors, frontend, controllers, services, repositories, and the MongoDB database for its corresponding use case.

Sequence diagrams include: activation bars to show participant lifetimes, synchronous message arrows with labels (method/HTTP calls), return dashed arrows, and alt/opt combined fragments for conditional flows. Diagrams are created using PlantUML. Insert exported diagram images in the placeholder sections below.

2. Use Case Summary

UC ID	Use Case Name	Primary Actor	Type	Priority
UC-01	User Registration	Student, Instructor	Major	High
UC-02	User Authentication (Login/Logout)	Student, Instructor	Major	High
UC-03	Course Creation	Instructor	Major	High
UC-04	Course Enrollment	Student	Major	High
UC-05	Content Upload & Management	Instructor	Major	High
UC-06	Learning Dashboard & Progress Tracking	Student	Minor	Medium

UC ID	Use Case Name	Primary Actor	Type	Priority
UC-07	Student Monitoring by Instructor	Instructor	Minor	Medium
UC-08	Search Courses	Student, Instructor	Minor	Medium
UC-09	User Profile Management	Student, Instructor	Minor	Low
UC-10	Contact & Support	Student, Instructor, Anonymous	Minor	Low

3. Sequence Diagram Specifications

1. UC-01 — User Registration

Use Case Reference	
Use Case ID	UC-01
Use Case Name	User Registration
Primary Actor(s)	Student, Instructor
Type	Major
Priority	High

Description: Shows the message flow from the user filling the registration form through the frontend, AuthController, AuthService, UserRepository, and MongoDB. Includes the alt fragment for duplicate email detection.

Key Interactions: *POST /api/auth/register → AuthController → AuthService → UserRepository → MongoDB. Alt fragment: email exists → 409 response; email new → hash + save + 201 response.*

Participants

Participant	Role
Student/Instructor	Initiates registration via browser
Next.js Frontend	Sends HTTP request to backend
AuthController	Entry point; routes to AuthService
AuthService	Business logic; calls UserRepository
UserRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — users collection

[Sequence Diagram — UC-01: User Registration]

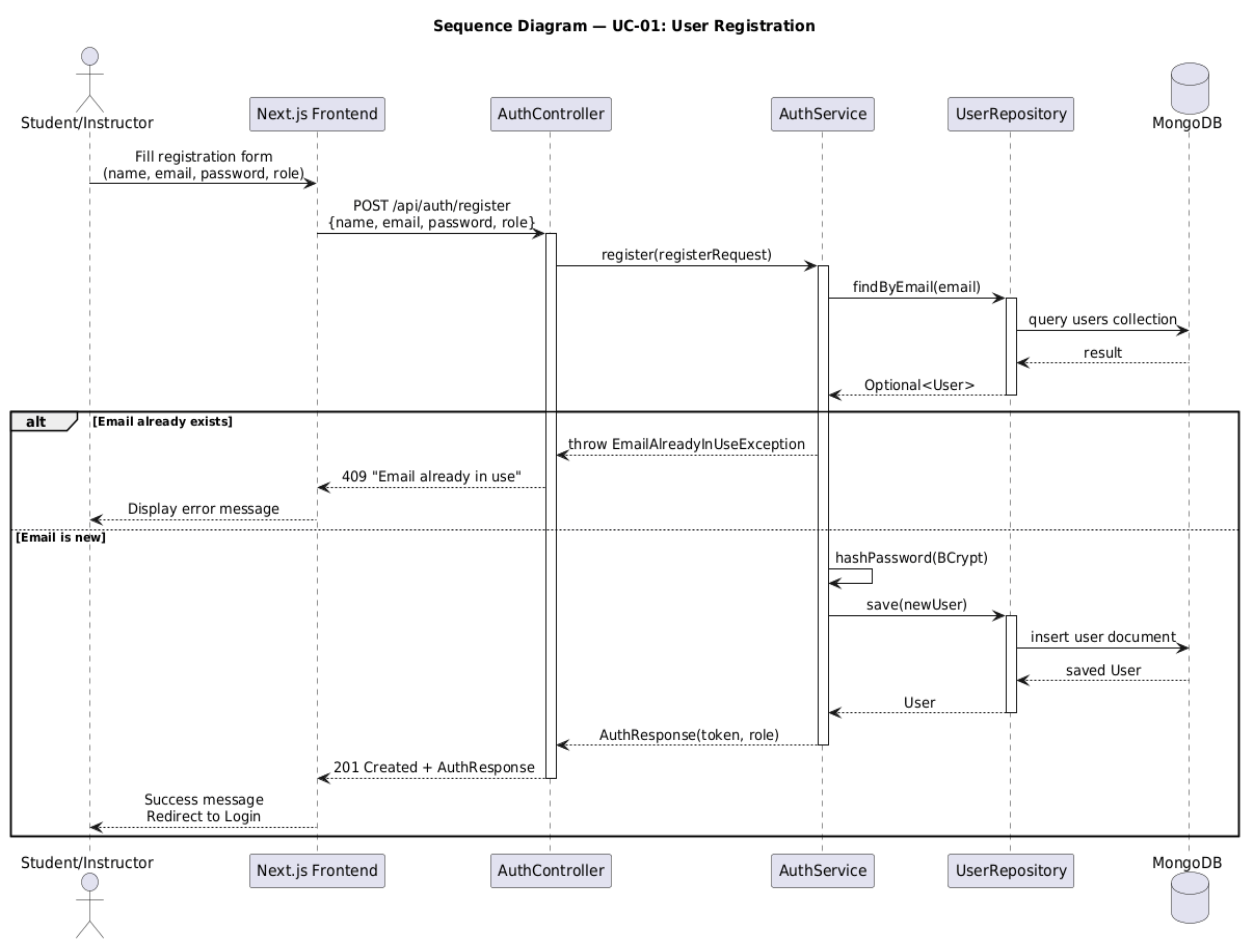

Figure 1: Sequence Diagram — UC-01: User Registration

2. UC-02 — User Authentication (Login/Logout)

Use Case Reference	
Use Case ID	UC-02
Use Case Name	User Authentication (Login/Logout)
Primary Actor(s)	Student, Instructor
Type	Major
Priority	High

Description: Shows two interaction frames: Login (credential validation, JWT generation) and Logout (client-side token removal). Includes alt fragment for invalid credentials.

Key Interactions: *Login: POST /api/auth/login → AuthController → AuthService → JwtService → JWT returned. Logout: frontend clears localStorage — no server call required.*

Participants

Participant	Role
Student/Instructor	Initiates login/logout
Next.js Frontend	Stores/clears JWT in localStorage
AuthController	Receives login request
AuthService	Validates credentials
JwtService	Generates signed JWT token
UserRepository	Looks up user by email
MongoDB	Persistence layer — users collection

[Sequence Diagram — UC-02: User Authentication (Login/Logout)]

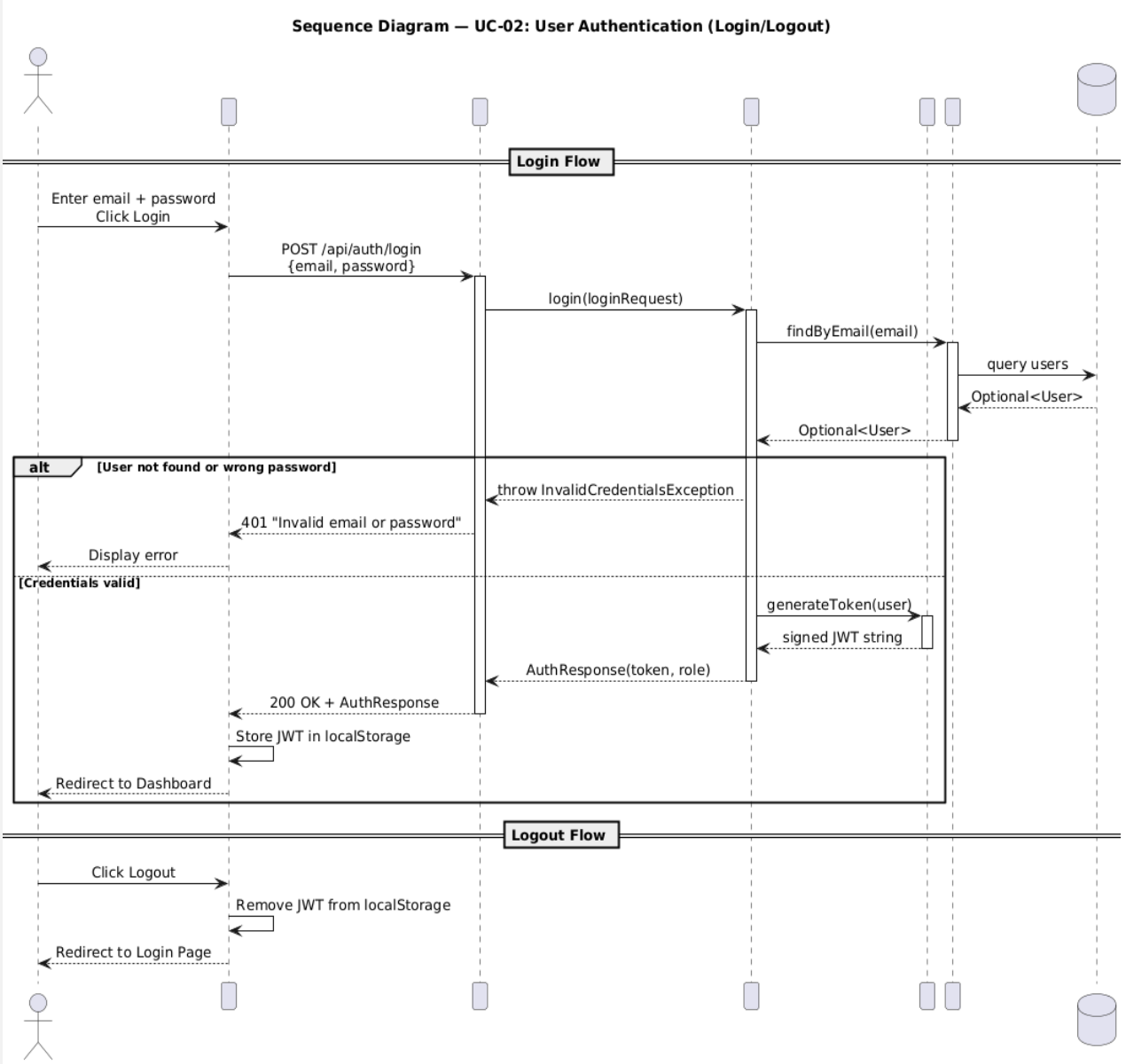

Figure 2: Sequence Diagram — UC-02: User Authentication (Login/Logout)

3. UC-03 — Course Creation

Use Case Reference	
Use Case ID	UC-03
Use Case Name	Course Creation
Primary Actor(s)	Instructor
Type	Major
Priority	High

Description: Depicts JWT validation, field validation, and course persistence. Alt fragments cover invalid token, missing fields, and DB save failure.

Key Interactions: *POST /api/courses → CourseController (validates JWT) → CourseService (validates fields) → CourseRepository → MongoDB insert → 201 response.*

Participants

Participant	Role
Instructor	Fills and submits course form
Next.js Frontend	Sends request with Bearer JWT
CourseController	JWT validation + routing
CourseService	Field validation + business logic
CourseRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — courses collection

[Sequence Diagram — UC-03: Course Creation]

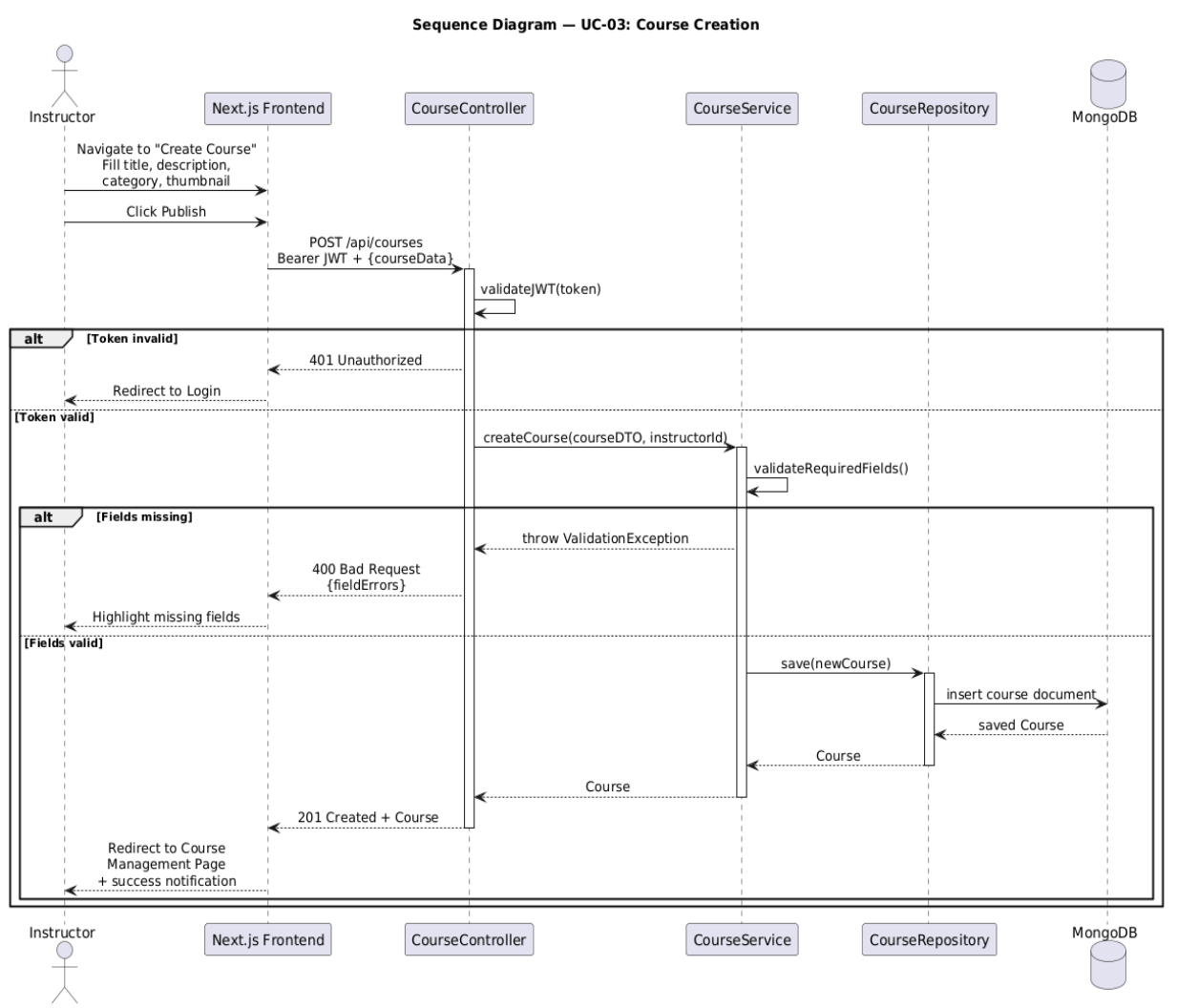

Figure 3: Sequence Diagram — UC-03: Course Creation

4. UC-04 — Course Enrollment

Use Case Reference	
Use Case ID	UC-04
Use Case Name	Course Enrollment
Primary Actor(s)	Student
Type	Major
Priority	High

Description: Shows course browsing, enrollment request, duplicate check via EnrollRepository, and enrollment document creation. Alt fragment for already-enrolled case.

Key Interactions: *GET /api/courses → course list. POST /api/enroll → EnrollController → EnrollService → duplicate check → save Enroll doc → 201 response.*

Participants

Participant	Role
Student	Browses and enrolls in course
Next.js Frontend	Renders courses; sends enroll request
EnrollController	JWT validation + routing
EnrollService	Duplicate check + enrollment logic
EnrollRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — enroll collection

[Sequence Diagram — UC-04: Course Enrollment]

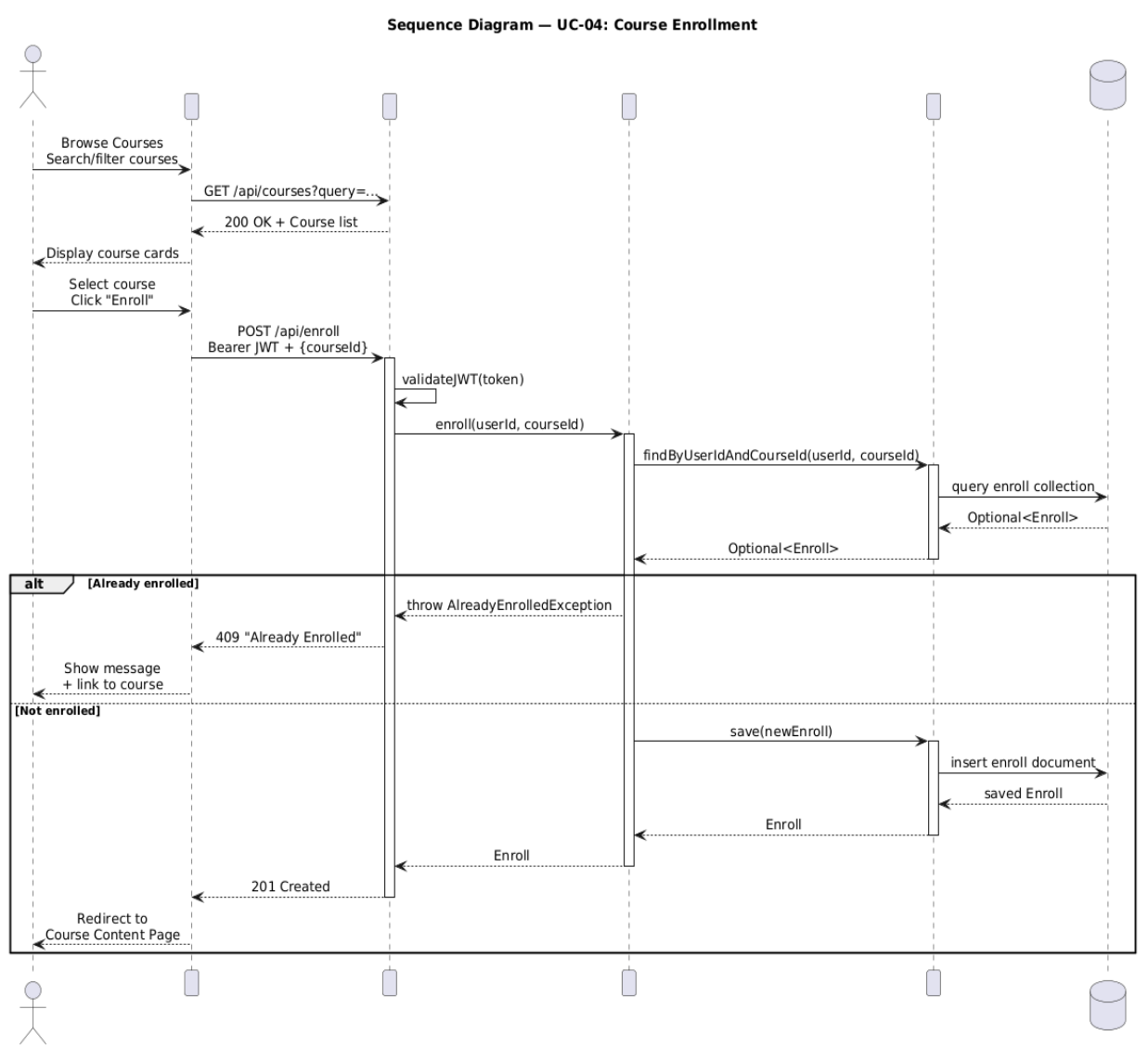

Figure 4: Sequence Diagram — UC-04: Course Enrollment

5. UC-05 — Content Upload & Management

Use Case Reference	
Use Case ID	UC-05
Use Case Name	Content Upload & Management
Primary Actor(s)	Instructor
Type	Major
Priority	High

Description: Spans three interaction frames: Add Lecture, Edit Lecture, and Delete Lecture. Each frame shows the full message chain from frontend to MongoDB.

Key Interactions: *Add: POST /api/lessons. Edit: PUT /api/lessons/{id}. Delete: DELETE /api/lessons/{id}. All via LessonController → LessonService → LessonRepository → MongoDB.*

Participants

Participant	Role
Instructor	Manages course lectures
Next.js Frontend	Sends CRUD requests
LessonController	Routes requests; validates JWT
LessonService	Validation + CRUD logic
LessonRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — lessons collection

[Sequence Diagram — UC-05: Content Upload & Management]

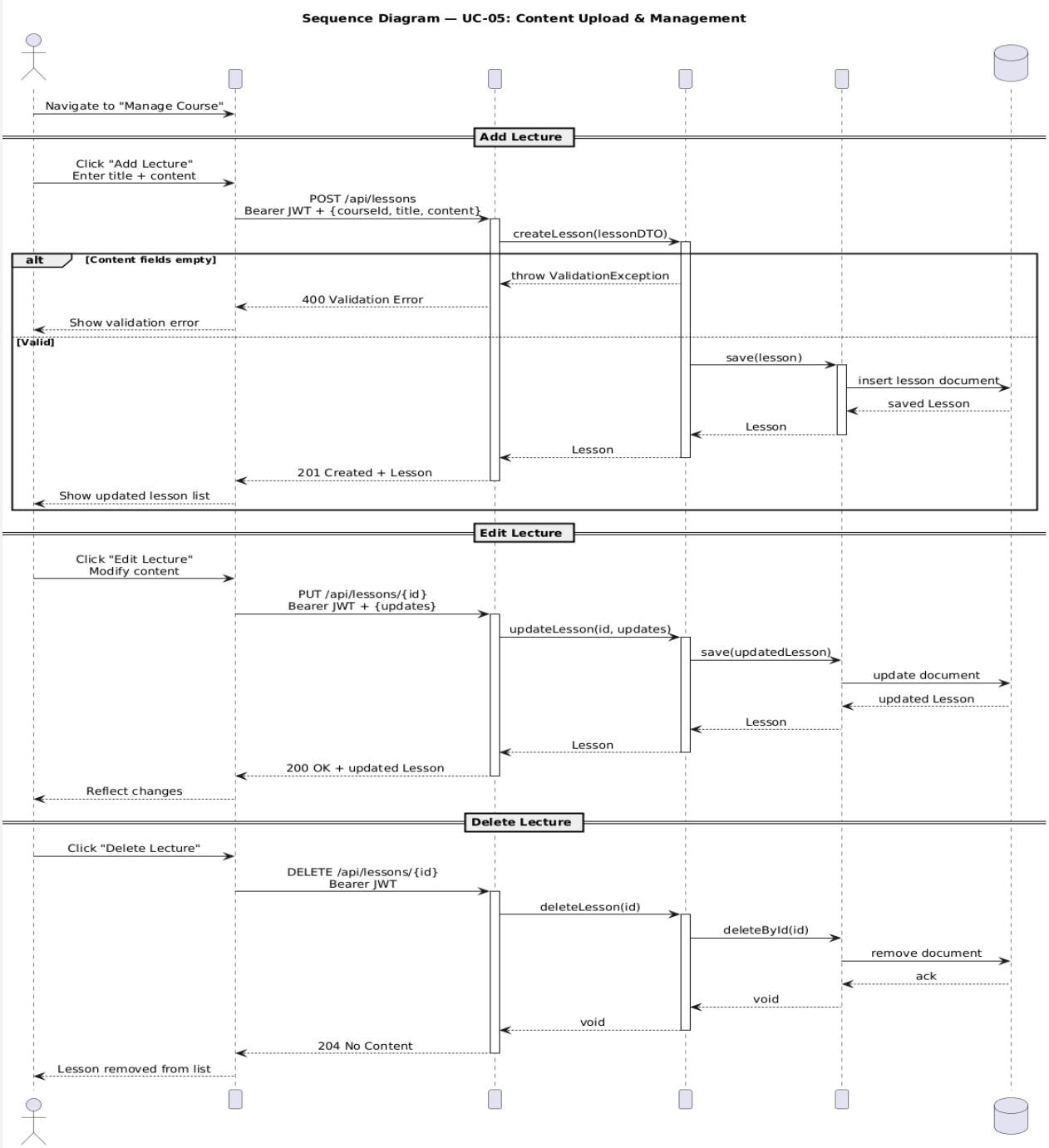

Figure 5: Sequence Diagram — UC-05: Content Upload & Management

6. UC-06 — Learning Dashboard & Progress Tracking

Use Case Reference	
Use Case ID	UC-06
Use Case Name	Learning Dashboard & Progress Tracking
Primary Actor(s)	Student
Type	Minor
Priority	Medium

Description: Shows dashboard load (enrollment fetch + progress fetch) and lesson access triggering a progress upsert. Two separate GET calls then a POST for marking lessons.

Key Interactions: *GET /api/enroll/my-courses → enrolled list. GET /api/progress/{courseId} → progress data. POST /api/progress/mark → upsert LessonProgress.*

Participants

Participant	Role
Student	Accesses dashboard and lessons
Next.js Frontend	Renders dashboard UI
EnrollController	Returns enrolled courses
ProgressController	Returns and updates progress
EnrollRepository	Queries enroll collection
LessonProgressRepository	Queries/upserts lesson_progress
MongoDB	Persistence layer

[Sequence Diagram — UC-06: Learning Dashboard & Progress Tracking]

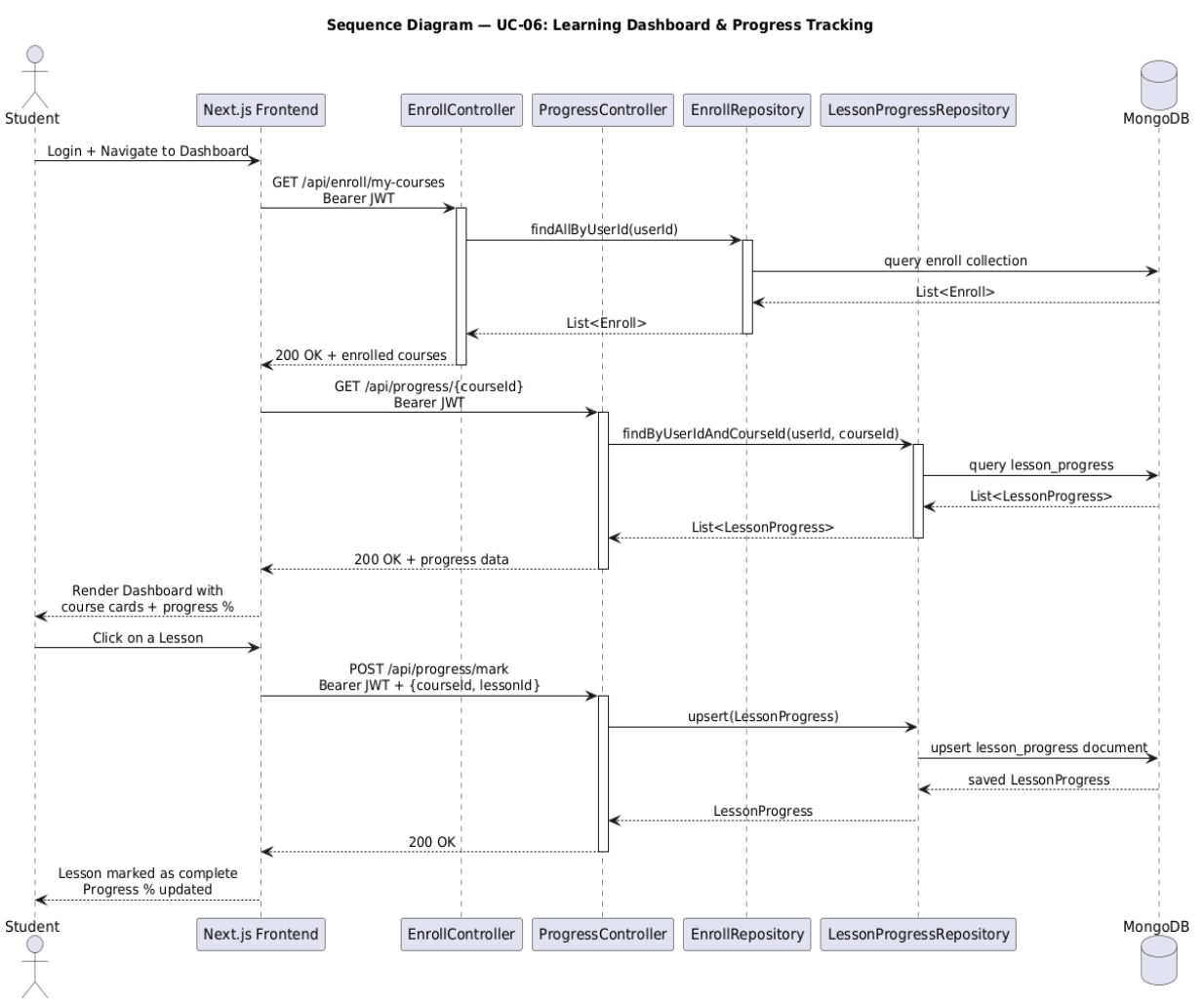

Figure 6: Sequence Diagram — UC-06: Learning Dashboard & Progress Tracking

7. UC-07 — Student Monitoring by Instructor

Use Case Reference	
Use Case ID	UC-07
Use Case Name	Student Monitoring by Instructor
Primary Actor(s)	Instructor
Type	Minor
Priority	Medium

Description: Shows ownership verification, enrollment retrieval, and progress aggregation for all students in a course. Alt fragment for empty enrollment and unauthorized access.

Key Interactions: *GET /api/courses/{courseId}/students → ownership check → EnrollRepository → LessonProgressRepository → StudentProgressDTO[] returned.*

Participants

Participant	Role
Instructor	Views student progress for a course
Next.js Frontend	Renders student list
CourseController	JWT + ownership validation
EnrollService	Aggregates enrollment + progress
EnrollRepository	Queries enroll collection
LessonProgressRepository	Queries lesson_progress collection
MongoDB	Persistence layer

[Sequence Diagram — UC-07: Student Monitoring by Instructor]

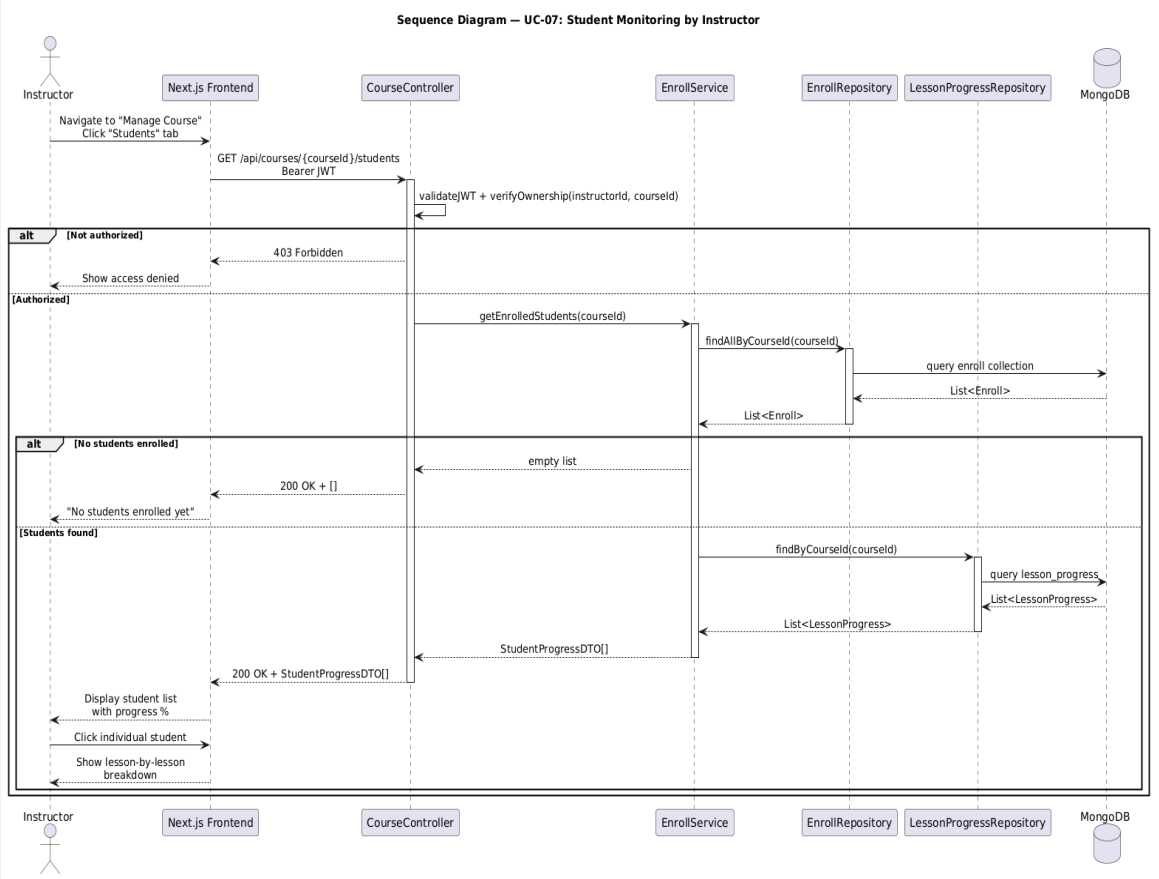

Figure 7: Sequence Diagram — UC-07: Student Monitoring by Instructor

8. UC-08 — Search Courses

Use Case Reference	
Use Case ID	UC-08
Use Case Name	Search Courses
Primary Actor(s)	Student, Instructor
Type	Minor
Priority	Medium

Description: Depicts query routing: empty search triggers findAll(); non-empty triggers regex search on title + description. Alt fragment for no-results case.

Key Interactions: *GET /api/courses?q={term} → CourseController → CourseService → CourseRepository (findAll or regex) → filtered course list returned.*

Participants

Participant	Role
Student/Instructor	Enters search term or filter
Next.js Frontend	Renders search UI and results
CourseController	Routes search request
CourseService	Builds query; calls repository
CourseRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — courses collection

[Sequence Diagram — UC-08: Search Courses]

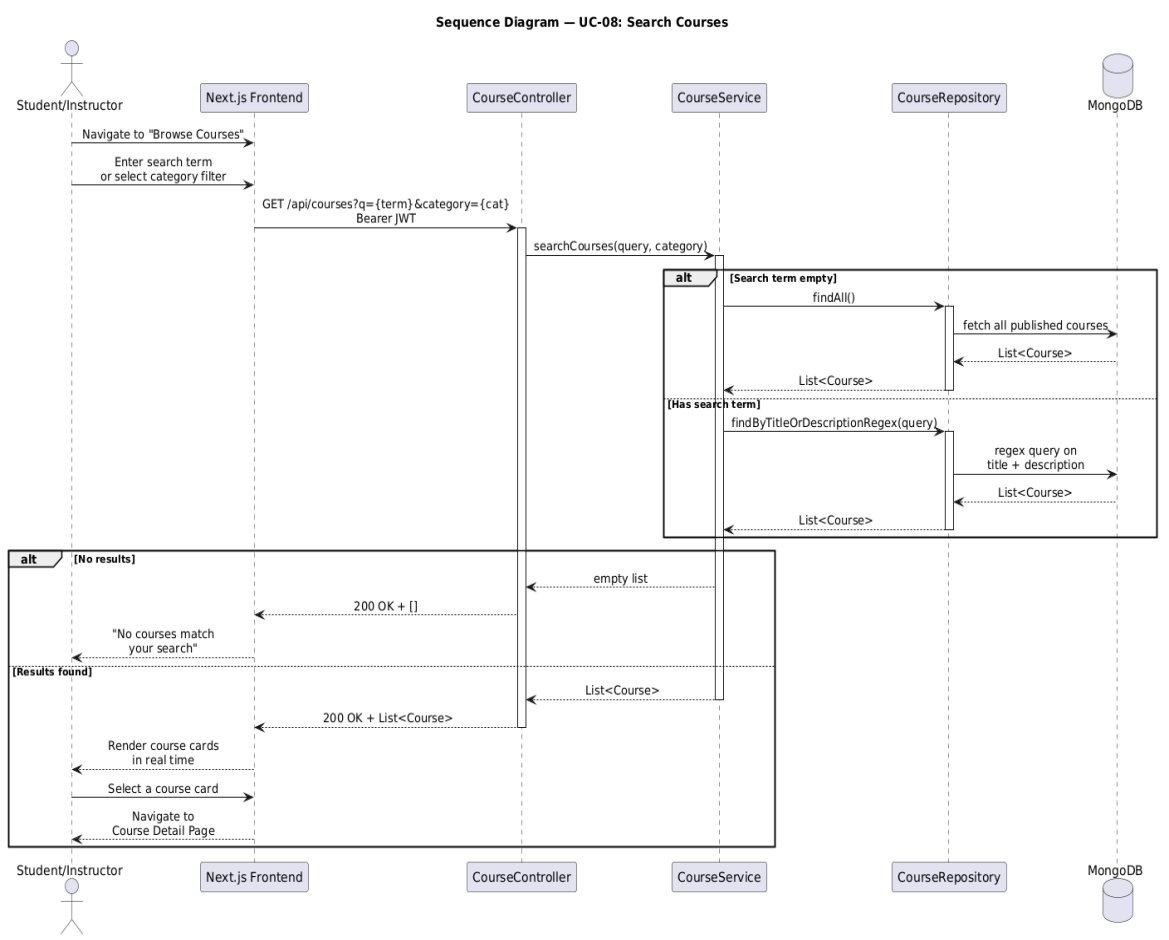

Figure 8: Sequence Diagram — UC-08: Search Courses

9. UC-09 — User Profile Management

Use Case Reference	
Use Case ID	UC-09
Use Case Name	User Profile Management
Primary Actor(s)	Student, Instructor
Type	Minor
Priority	Low

Description: Shows profile fetch, field update with validation, password policy check, email conflict detection, and MongoDB update. Multiple alt fragments for error cases.

Key Interactions: *GET /api/users/profile → UserDTO. PUT /api/users/profile → UserController → UserService (validate) → UserRepository → updated UserDTO.*

Participants

Participant	Role
Student/Instructor	Views and updates profile
Next.js Frontend	Renders profile form
UserController	JWT validation + routing
UserService	Validation + update logic
UserRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — users collection

[Sequence Diagram — UC-09: User Profile Management]

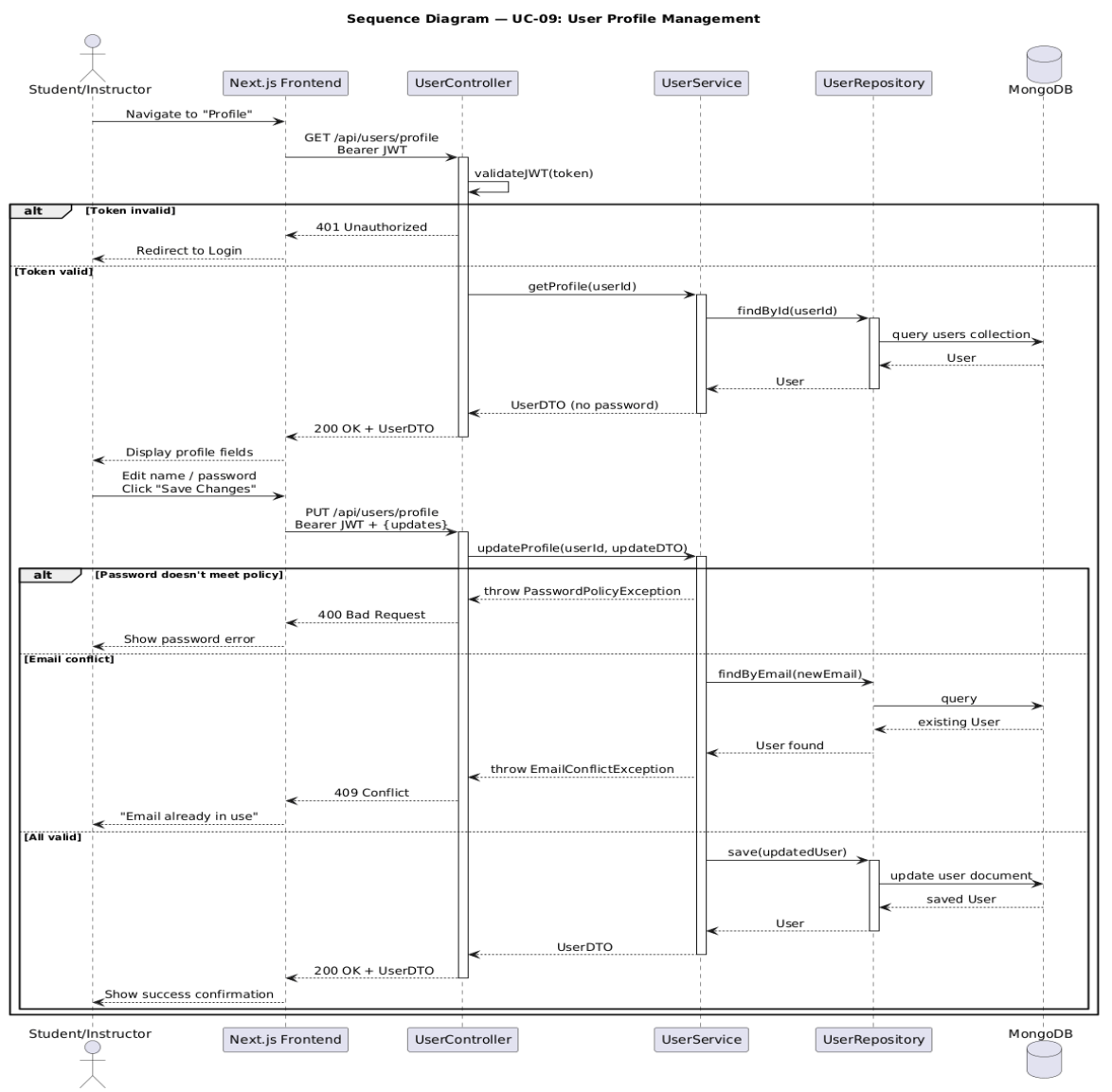

Figure 9: Sequence Diagram — UC-09: User Profile Management

10. UC-10 — Contact & Support

Use Case Reference	
Use Case ID	UC-10
Use Case Name	Contact & Support
Primary Actor(s)	Student, Instructor, Anonymous
Type	Minor
Priority	Low

Description: Shows form submission without authentication, field validation, ContactMessage persistence to MongoDB, and success acknowledgement. Alt fragment for validation failure.

Key Interactions: *POST /api/contact (no JWT) → ContactController → ContactService (validate) → ContactRepository → MongoDB insert → 201 + success message.*

Participants

Participant	Role
User (Any/Anonymous)	Submits support message
Next.js Frontend	Renders contact form; shows result
ContactController	Receives unauthenticated request
ContactService	Validates fields + saves message
ContactRepository	Spring Data MongoDB interface
MongoDB	Persistence layer — contact_messages

[Sequence Diagram — UC-10: Contact & Support]

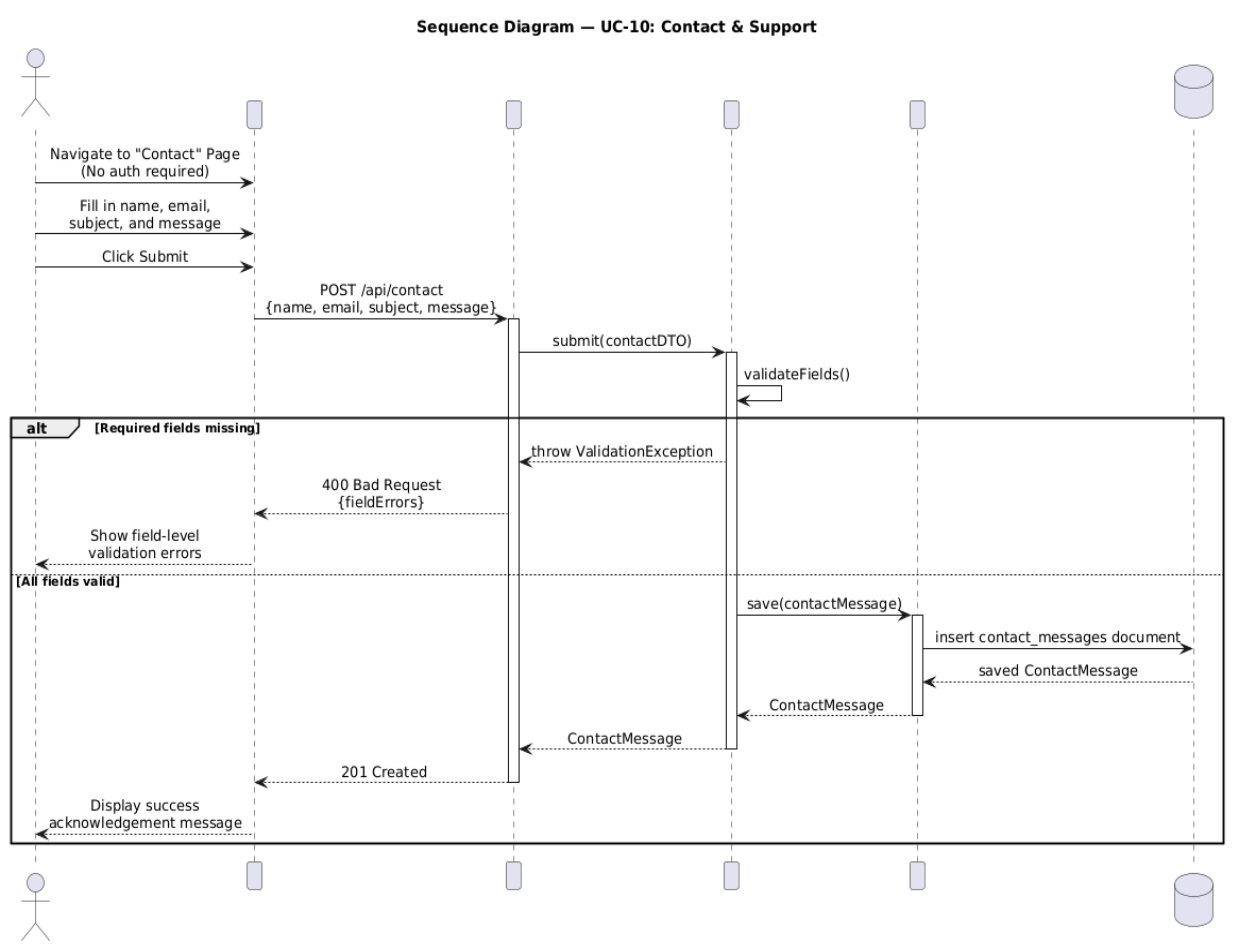

Figure 10: Sequence Diagram — UC-10: Contact & Support